3. 합의 및 검증 프로세스

개요

Sapphire ParaTime에서 TEE 실행 결과의 합의와 검증이 어떻게 이루어지는지,

그리고 분산 환경에서의 신뢰성 보장 메커니즘을 상세히 설명합니다.

3.1 합의 아키텍처

계층적 합의 구조

```
┌─────────────────────────────────────────────────────────────┐
│                    Oasis Consensus Layer                    │
│ ┌──────────────────────────────────────────────────────────┐ │
│ │                     Validator Nodes                      │ │
│ │ ┌────────────────────────────────────────────────────┐ │ │
│ │ │                  Sapphire ParaTime                 │ │ │
│ │ │ ┌──────────────────────────────────────────────┐ │ │ │
│ │ │ │              TEE Execution Nodes             │ │ │ │
│ │ │ │               - SGX Enclave 1                │ │ │ │
│ │ │ │               - SGX Enclave 2                │ │ │ │
│ │ │ │               - SGX Enclave N                │ │ │ │
│ │ │ └──────────────────────────────────────────────┘ │ │ │
```

```
│ │ └──────────────────────────────────────────────────────┘ │ │
│ └──────────────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────────────┘
```

합의 모듈 구성

```
modules::consensus::Genesis {
parameters: modules::consensus::Parameters {
gas_costs: modules::consensus::GasCosts {
round_root: 10_000
},
consensus_denomination: Denomination::NATIVE,
consensus_scaling_factor: 1_000_000_000, // 18 소수점으로 스케일링
min_delegate_amount: 100_000_000_000, // 최소 위임 금액
},
}
```

3.2 SGX 증명 (Attestation) 프로세스

원격 증명 단계

1. **엔클레이브 초기화**

- SGX 엔클레이브 생성 및 측정값(MRENCLAVE) 계산
- 서명 키 생성 및 증명서 요청

2. **Intel Attestation Service (IAS) 검증**

- 엔클레이브 Quote 생성
- IAS를 통한 하드웨어 검증
- 증명서(Attestation Report) 수신

3. **네트워크 등록**

- 검증된 증명서를 네트워크에 제출
- 다른 노드들의 증명서 검증
- 신뢰할 수 있는 실행 노드로 등록

신뢰 루트 검증

```
#[cfg(target_env = "sgx")]
fn consensus_trust_root() -> Option<TrustRoot> {
if is_testnet() {
Some(TrustRoot {
height: 24544750,
hash: "6b2b5e7990e0c8bee8035934bc52a83a16fe4bff4dbb81b1aa5a8f9409b2eafe".into(),
runtime_id: "0000000000000000000000000000000000000000000000000a6d1e3ebf60dff6c".i
chain_context: "0b91b8e4e44b2003a7c5e23ddadb5e14ef5345c0ebcb3ddcae07fa2f244cab76
})
} else {
// 메인넷 신뢰 루트
Some(TrustRoot {
height: 22327937,
```

```
hash: "01670e0af13fa4ceaeee8bd9672374d27a889b9304c44ab9acfd7f97663ee1b9".into(),

runtime_id: "0000000000000000000000000000000000000000000000000f80306c9858e7279".i

chain_context: "bb3d748def55bdfb797a2ac53ee6ee141e54cd2ab2dc2375f4a0703a178e6e55

})

}

}
```

3.3 블록 생성 및 검증 프로세스

블록 생성 단계

1. **트랜잭션 수집**

- 메모리 풀에서 트랜잭션 선택
- 가스 가격 및 우선순위에 따른 정렬
- 배치 크기 제한 적용

2. **TEE 내부 실행**

- 암호화된 트랜잭션 복호화
- EVM에서 스마트 컨트랙트 실행
- 상태 변경사항 계산

3. **실행 결과 암호화**

- 상태 변경사항 암호화
- 실행 증명(Execution Proof) 생성
- 블록 헤더 생성

검증 프로세스

```
// 배치 처리 설정
const SCHEDULE_CONTROL: config::ScheduleControl = config::ScheduleControl {
initial_batch_size: 50,
batch_size: 50,
min_remaining_gas: 1_000,
max_tx_count: 1_000,
};
```

3.4 분산 검증 메커니즘

다중 TEE 검증

1. **병렬 실행**
   - 여러 TEE 노드에서 동일한 트랜잭션 배치 실행
   - 각 노드는 독립적으로 실행 결과 계산
2. **결과 비교**
   - 실행 결과의 해시값 비교
   - 상태 루트 해시 검증
   - 불일치 시 재실행 또는 노드 제외
3. **합의 도달**
   - 2/3 이상의 노드가 동일한 결과 도출 시 합의
   - 비잔틴 장애 허용 (BFT) 알고리즘 적용

상태 일관성 검증

```
// 상태 마이그레이션을 통한 일관성 보장
fn migrate_state<C: sdk::Context>(_ctx: &C) {
let genesis = Self::genesis_state();
// 각 모듈의 파라미터 동기화
modules::core::Module::<Config>::set_params(genesis.0.parameters);
modules::accounts::Module::set_params(genesis.1.parameters);
modules::consensus::Module::set_params(genesis.2.parameters);
// ...
}
```

3.5 키 관리 및 보안

신뢰할 수 있는 서명자 관리

```
fn trusted_signers() -> Option<TrustedSigners> {
let tps = keymanager::trusted_policy_signers();
Some(TrustedSigners {
signers: tps.signers.into_iter().map(|s| PublicKey(s.0)).collect(),
threshold: tps.threshold,
})
}
```

키 관리자 통합

- **분산 키 관리**: 키 관리자를 통한 암호화 키 분산 관리
- **키 회전**: 정기적인 암호화 키 갱신
- **접근 제어**: 권한 있는 노드만 키 접근 가능

3.6 장애 복구 메커니즘

노드 장애 처리

1. **장애 감지**

- 하트비트 메커니즘을 통한 노드 상태 모니터링
- 응답 시간 초과 시 장애로 판단

2. **자동 복구**

- 장애 노드 자동 제외
- 백업 노드 활성화
- 상태 동기화 및 서비스 재개

3. **데이터 복구**

- 암호화된 상태 백업에서 복구
- 다른 노드로부터 상태 동기화

네트워크 분할 처리

- **분할 감지**: 네트워크 연결성 모니터링
- **합의 유지**: 과반수 노드가 연결된 파티션에서 합의 지속
- **재결합**: 네트워크 복구 시 상태 동기화

3.7 성능 최적화

병렬 검증

```
// 병렬 처리를 위한 배치 설정
max_batch_gas: 15_000_000,
max_tx_size: 128 * 1024,
max_tx_signers: 3,
```

검증 최적화

- **증분 검증**: 변경된 상태만 검증
- **캐시 활용**: 이전 검증 결과 캐싱
- **파이프라인**: 검증과 실행의 파이프라인 처리

3.8 보상 시스템

검증자 보상

```
modules::rewards::Genesis {
parameters: modules::rewards::Parameters {
schedule: modules::rewards::types::RewardSchedule {
steps: vec![modules::rewards::types::RewardStep {
until: 27_500,
amount: BaseUnits::new(3_000_000_000_000_000_000, Denomination::NATIVE),
```

```
}],

},

participation_threshold_numerator: 3,

participation_threshold_denominator: 4,

},

}
```

참여 인센티브

- **참여율 기반 보상**: 75% 이상 참여 시 보상 지급
- **성능 기반 보상**: 빠르고 정확한 검증에 추가 보상
- **슬래싱**: 악의적 행동 시 스테이크 삭감

3.9 모니터링 및 감사

실시간 모니터링

- **성능 메트릭**: 처리량, 지연시간, 오류율 모니터링
- **보안 이벤트**: 비정상적인 활동 감지 및 알림
- **리소스 사용량**: CPU, 메모리, 네트워크 사용량 추적

감사 로그

- **실행 로그**: 모든 트랜잭션 실행 기록
- **합의 로그**: 합의 과정 및 결과 기록
- **보안 로그**: 인증, 권한 부여 이벤트 기록

결론

Sapphire의 합의 및 검증 프로세스는 SGX 증명을 통한 하드웨어 신뢰성과 분산 합의 알고리즘을 결합하여 높은 수준의 보안과 가용성을 제공합니다. 다중 TEE 검증과 비잔틴 장애 허용을 통해 악의적인 노드나 하드웨어 장애에도 불구하고 시스템의 무결성을 보장합니다.